A3: LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Team guide: the whole project so far, what to install, how to build, run and test, and how we work together from now on

14 October 2026

Contents

1. How we work from now on (and why) **3**

2. The project in one page **3**

3. Repository map **4**

4. What has been done so far **5**

4.1 Stage 1 as found . . . 5

4.2 Gaps fixed . . . 5

4.3 The differential plan oracle (new) . . . 6

4.4 Verified on a real server . . . 6

4.5 Not done yet . . . 6

5. What to install **6**

5.1 With sudo (recommended, fastest) . . . 6

5.2 Without sudo . . . 7

5.3 Python environment . . . 7

5.4 Optional . . . 7

6. Build and start PostgreSQL **7**

6.1 Assert-enabled `master` (for bug hunting) - about 10-15 minutes . . . 7

6.2 Coverage build (for the three-way experiment) . . . 7

6.3 Docker alternative . . . 8

6.4 Keep WSL alive . . . 8

7. How to test **8**

8. How to run a fuzzing experiment (works today with the random baseline) **8**

9. What to send back (keep it short) **9**

10. Plan and timeline **9**

11. Known pitfalls **10**

1. How we work from now on (and why)

Short answer to "are we wasting tokens?": partly, yes. About half of the effort so far went into *waiting for commands*. Each WSL call boots the Linux VM (about 30 seconds), the PostgreSQL build took 13 minutes, and one self-test took 5 minutes because of a network stall. Waiting costs tokens and gives no insight. That time is better spent on design, code and analysing results.

The working model from now on:

Who	Does what
You (the team)	Install tools, build PostgreSQL, start servers, run tests and fuzzing campaigns (minutes to hours), and paste back the **short outputs** listed in Section 9.
Claude	Writes and fixes code (grammar, generators, LLM loop, oracles, minimizer, triage), reads your outputs, decides what to fix or fuzz next, analyses bug candidates, drafts upstream bug reports, and writes the PDFs and paper text.

Claude will still run a command itself when it is quick and the answer is needed immediately (for example unit tests after a code change). Anything long (builds, campaigns, minimization of many crashes) is yours.

Rule for pasting outputs: paste the *summary*, not the raw log. Every command in this guide says exactly what to paste. Large files (`events.jsonl`, server logs) should never be pasted whole.

2. The project in one page

Goal. Build a fuzzer that generates SQL for PostgreSQL, runs it on a specially built server, and detects bugs. Then (1) compare three generator strategies scientifically, and (2) find at least one **real, previously unknown PostgreSQL bug that maintainers accept** (needed for full marks on the 50 % implementation component).

The three arms of the study (same time budget, same server, same schema):

```
Random/Mutation  ->  Grammar-only  ->  Grammar + LLM
```

- **Random/Mutation:** random SQL tokens, or seed statements with random token edits. Most of it is rejected by the parser. This is the baseline.
- **Grammar-only:** SQL generated from a typed PostgreSQL grammar that knows the schema, so almost every statement parses and many run. It reaches deeper code (planner, executor).
- **Grammar + LLM:** the same, plus an LLM that (a) proposes grammar edits for SQL features we have not exercised and (b) writes targeted test cases for new or rarely tested PostgreSQL features.

How a bug is detected (the "oracle"). Each statement is classified from the server's answer and the server log:

Outcome	How it is detected	Bug?
syntax / semantic / runtime error, timeout	SQLSTATE class	no
`internal_error`	SQLSTATE `XX000` ("cache lookup failed", "unrecognized node type", ...)	**yes**
`assertion`	`TRAP: failed Assert(...)` in the log (needs an assert-enabled build)	**yes**
`crash`	lost connection / "terminated by signal"	**yes**
`sanitizer`	AddressSanitizer / UBSan report in the log	**yes**
`wrong_result`	same query, different rows under another planner setting	**yes**
`panic`, `data_corruption`	`PANIC:`, `XX001`/`XX002`	**yes**

Why this can find new bugs. We fuzz the newest PostgreSQL code (`master`, currently `20devel`) built with assertions. New code has the most undiscovered bugs, and assertions catch problems that would otherwise pass silently. The wrong-result oracle catches optimizer bugs that never crash.

Grading (for orientation): implementation and results 50 % (full marks need an upstream-accepted bug), meetings and presentation 25 % (everyone must be able to explain everything), report in ACM/FSE format 25 %.

3. Repository map

```
harness/                        Member A: the instrumented server
  docker/Dockerfile             build PostgreSQL from git (cassert, debug, coverage, sanitizers)
  docker/entrypoint.sh          container start: initdb + server log to /logs
  docker/postgresql.fuzz.conf   fuzzing server settings
  docker/coverage.sh            gcov/lcov coverage as one JSON line (+ reset)
  docker/gdb_backtrace.sh       crash backtrace via single-user mode under gdb
  pgctl.py                      manage Docker workers from the host
  local/build_pg.sh             build PostgreSQL without Docker and without root
  local/pg_local.sh             manage N local servers (init/start/stop/coverage/backtrace)
  executor.py                   Stage 1 single-query tool (psql)
src/                            the fuzzer "pgfuzz" (Python)
  pgfuzz/core/                  data types and plug-in interfaces
  pgfuzz/eval/                  oracle, signatures, features, metrics, differential oracle, stats
  pgfuzz/reference/             PostgresExecutor, MockExecutor, Random/Corpus generators
  pgfuzz/runner/                experiment runner, trials, selftest
  pgfuzz/runlog/, report/       log format, reports, tables, plots
  configs/                      smoke.yaml (simulated), stage1.yaml (real), local.yaml (dev)
  corpus/                       schema.sql (shared database state), seeds/
  known_bugs/                   catalog for the known-bug reproduction metric
  tests/                        unit and end-to-end tests
```

```
paper/                          the FSE-format report (LaTeX)
docs/                           these guides (Markdown + PDF)
```

4. What has been done so far

4.1 Stage 1 as found

Member C's evaluation framework was complete and well tested (oracle, deduplication, 148-feature SQL catalog, metrics, log format, runner, statistics, report generator, 58 tests). Member A's harness built PostgreSQL 18.6 with coverage only. Member B's generator was a stand-in.

4.2 Gaps fixed

#	Gap	Fix
1	No `--enable-cassert`: assertion bugs invisible	All builds use assertions and debug symbols by default
2	Fixed old release	Build any git ref; default `master`
3	Oracle could not read the server log	Log written to a known file / mounted `/logs`
4	Coverage counters never reset between trials	`coverage_reset_command` runs at the start of every trial
5	No coverage command	`coverage.sh` prints JSON, stable denominator
6	lcov time charged to the fuzzing budget	Coverage time excluded, reported as `coverage_overhead_s`
7	Hard-coded `/home/gow/...` paths	Environment variables
8	Crash-only oracle	New **differential plan oracle** (`wrong_result`) and sanitizer detection
9	`'infinity'::date` etc. misreported as errors (client decoding)	Results loaded as text
10	Result list shifted after a failed `RELEASE SAVEPOINT`	Fixed
11	Duplicate crashes counted as different bugs	C backtraces parsed from the log, used for "stack" signatures
12	Windows: `localhost` stalls ~2 min (IPv6 first)	Configs use `127.0.0.1`
13	Reading the log over the WSL share costs ~40 ms per test case	Log read only when a test case fails (crash evidence always comes with a failure)

4.3 The differential plan oracle (new)

PostgreSQL planner settings change *how* a query runs, never *what* it returns. For every successful, deterministic query, the executor re-runs it under up to 2 of 17 **plan variants** (nested loop only, hash join only, merge join only, no hash aggregation, index scans only, forced parallel query, partition-wise join, no partition pruning, GEQO, tiny `work_mem`, no self-join elimination, JIT, ...) inside a savepoint that is rolled back, and compares the result multisets. Queries whose result is not determined by the data (`LIMIT`, `random()`, `array_agg` without `ORDER BY`, `row_number()`, ...) are skipped, and float columns are masked when the query aggregates, so false alarms are rare. A difference is a `wrong_result` bug candidate. An internal error or crash under a variant is also reported, together with the variant's settings.

4.4 Verified on a real server

On this laptop (WSL Ubuntu 24.04, no sudo), `harness/local/build_pg.sh` built PostgreSQL **20devel at commit `d733fca`** with `--enable-cassert --enable-debug --with-libxml`. Two servers run on ports 5432 and 5433. `python -m pgfuzz selftest configs/local.yaml` passed **10/10 checks** (SQLSTATE mapping, continuing after errors, timeouts, no state leaks, orderly termination, reconnecting). The existing 58 unit tests pass.

Measured executor speed for 20 small test cases (2 statements each), Windows Python to the WSL server: 0.15 s without server-log reading, 1.0 s with it (before the fix in row 13). Starting the executor took 131 s with `localhost` and 0.2 s with `127.0.0.1`.

4.5 Not done yet

- The grammar language, grammar engine and PostgreSQL grammar (Stage 2).
- The LLM-guided generator and grammar updater (Stage 3).
- Minimizer, triage and bug-report drafting.
- Configs `stage2.yaml`, `stage3.yaml`, `hunt.yaml`; tests for the new parts.
- The Docker image has not been built yet (no Docker on this laptop).
- No experiment numbers and no bugs yet.

`configs/local.yaml` already names the grammar generator, which does not exist yet. `selftest` and `replay` work with it; `run` will work once the grammar lands.

5. What to install

Use **Linux or WSL2 Ubuntu** for the server. The fuzzer itself (Python) runs on Linux, WSL or Windows.

5.1 With sudo (recommended, fastest)

```
sudo apt-get update
sudo apt-get install -y build-essential git bison flex perl pkg-config \
  libreadline-dev zlib1g-dev libxml2-dev libxslt1-dev libicu-dev libssl-dev \
  liblz4-dev libzstd-dev lcov gdb python3 python3-venv python3-pip
```

5.2 Without sudo

`harness/local/build_pg.sh` builds m4, bison and flex into `~/pgfuzz/tools` automatically. It needs only `gcc`, `make`, `perl`, `git` and `curl`.

5.3 Python environment

Inside Linux/WSL (recommended, because the server log is then a local file):

```
cd <repo>/src
python3 -m venv ~/pgfuzz-venv && source ~/pgfuzz-venv/bin/activate
pip install -r requirements.txt        # PyYAML, psycopg[binary], matplotlib, pytest, scipy
```

On Windows: `cd src` then `pip install -r requirements.txt`.

5.4 Optional

- **Docker** (for parallel coverage experiments, one container per worker).
- **An LLM API key** for Stage 3: any OpenAI-compatible provider (OpenAI, OpenRouter, Groq, Together, DeepSeek, Gemini's OpenAI endpoint), a local Ollama, or Anthropic. It will be read from an environment variable, e.g. `export OPENAI_API_KEY=...`. Do not commit keys.

6. Build and start PostgreSQL

Important on WSL: the repository path contains spaces and `(1)`. Always quote it. Also, scripts edited on Windows may have CRLF line endings. Run `sed -i 's/\r$//' harness/local/*.sh harness/docker/*.sh` once if bash complains about `$'\r'`.

6.1 Assert-enabled `master` (for bug hunting) - about 10-15 minutes

```
cd "<repo>"
PG_REF=master bash harness/local/build_pg.sh
bash harness/local/pg_local.sh init 2      # 2 servers: ports 5432, 5433
bash harness/local/pg_local.sh start 2
bash harness/local/pg_local.sh env 2       # prints version, commit, log paths
```

6.2 Coverage build (for the three-way experiment)

```
PG_REF=REL_18_STABLE COVERAGE=1 bash harness/local/build_pg.sh
PG_BIN=~/pgfuzz/install/REL_18_STABLE-cov/bin ROOT=~/pgfuzz-cov BASE_PORT=5532 \
  bash harness/local/pg_local.sh init 1
# start the same way; coverage:  ... pg_local.sh coverage 0   (JSON line)
```

`pg_local.sh` uses the *newest* build under `~/pgfuzz/install` unless `PG_BIN` is set. Once both builds exist, always set `PG_BIN` explicitly (`~/pgfuzz/install/master/bin` for bug hunting).

Locally, all servers of one build share one set of gcov counters, so run coverage experiments with `-j 1`, or use Docker for parallel workers.

6.3 Docker alternative

```
python harness/pgctl.py build --ref master            # assert build
python harness/pgctl.py build --ref REL_18_STABLE --coverage
python harness/pgctl.py up --workers 4 --image pgfuzz:master
python harness/pgctl.py info --worker 0
```

6.4 Keep WSL alive

WSL shuts the VM down when no terminal is open, which stops the servers. Keep one WSL terminal open during campaigns, and re-run `pg_local.sh start 2` after a reboot.

7. How to test

Run from `src/`. "Paste" means: send this back to Claude.

Step	Command	Expected	Paste
Unit tests	`python -m pytest -q`	all pass	last 3 lines
Pipeline smoke test (no server)	`python -m pgfuzz run configs/smoke.yaml`	report path printed	nothing unless it fails
Executor acceptance	`python -m pgfuzz selftest configs/local.yaml`	`10/10 checks passed`	the PASS/FAIL lines
Replay a SQL file	`python -m pgfuzz replay configs/local.yaml file.sql --repeat 3`	JSON verdict per run	the JSON lines
Feature coverage of SQL	`python -m pgfuzz features corpus/seeds/basic.sql --missing`	e.g. `NN/148`	the summary line

Before `selftest`/`replay`, edit `configs/local.yaml`:

- `dsn`: keep `host=127.0.0.1`; the port comes from `base_port`.
- `server_log`: if Python runs **inside WSL/Linux**, use `/home/<user>/pgfuzz/data/pg-{worker}/postgresql.`
  if Python runs on **Windows**, use `//wsl.localhost/Ubuntu/home/<user>/pgfuzz/data/pg-{worker}/postgr`

8. How to run a fuzzing experiment (works today with the random baseline)

```
cd src
# configs/stage1.yaml: set server_log (and coverage commands for a coverage build)
python -m pgfuzz run configs/stage1.yaml --trials 1 -j 1      # 1 trial = 1 hour by default
python -m pgfuzz report ../results/stage1                      # rebuild the report
```

Output: `results/stage1/<arm>/trial-NN/` with `summary.json`, `failures/<signature>/testcase.sql`, and `results/stage1/report/report.md`. For a quick check, shorten `budget.seconds` to 300.

When a bug candidate appears (`failures/<sig>/`):

```
python -m pgfuzz replay configs/local.yaml ../results/.../failures/<sig>/testcase.sql --repeat 3
bash harness/local/pg_local.sh backtrace 0 ../results/.../failures/<sig>/testcase.sql   # crashes
```

9. What to send back (keep it short)

Situation	Paste
A command failed	the command, the last 30 lines of output
After an experiment	`results/<exp>/report/report.md` (it is short)
A bug candidate	`failure.json` *without* the `server_log` and `statements` fields, `testcase.sql`, and the replay JSON
A crash	the backtrace from `pg_local.sh backtrace` and 20 log lines around `TRAP:`
Server info	output of `pg_local.sh env 2`

10. Plan and timeline

When	Milestone	Owner of the running
Now	Claude writes the grammar language, engine, PostgreSQL grammar, grammar generator	Claude
Then	You run unit tests + a 5-minute grammar campaign on `master`; paste `report.md`	Team
Then	Claude tunes the grammar from the error statistics (target: most statements parse, many run)	Claude
Before meeting 2 (26-30 Oct)	LLM loop, minimizer, triage; first long campaigns on `master`; first bug candidates	Both
Before meeting 3 (2-6 Nov)	Triage, minimize, bisect, verify on latest release and `master`; file reports to pgsql-bugs; full 10-trial experiment on the coverage build	Team runs, Claude analyses and drafts reports
12-13 Nov	Paper and presentation from the measured numbers	Both

Start bug hunting early. Upstream acceptance takes days to weeks, and the deadline is 13 November. The first long campaigns on `master` should run as soon as the grammar generator exists, in parallel with the experiments.

11. Known pitfalls

- **Use `127.0.0.1`**, not `localhost`, on Windows.
- **`backtrace_on_internal_error`** does not exist in current `master`; the scripts only enable it if the server knows it. Do not add it by hand.
- **CRLF line endings** break bash scripts: `sed -i 's/\r$//' <script>`.
- **WSL VM shutdown** stops the servers; keep a terminal open.
- **Never report numbers from `configs/smoke.yaml`**; it uses a simulated server.
- **Every number in the paper must come from a real run** (`paper/STYLE.md`).